기존 D3-G Yocto 환경에 ROS 2를 추가하고 FWDN으로 기록하기

작성: 2026-10-28. 이전 수업에서 TOPST D3-G 이미지를 빌드한 환경을 재사용한다. **현재 실측 상태: ROS 2 빌드 성공(8,354 tasks, 종료 코드 0), 이미지 정적 검사·FWDN ZIP 검사 통과. 보드가 없어 실제 FWDN 기록·부팅은 미실행.** 이 문서에서 빌드 호스트, 다운로드 PC, D3 보드를 구분한다.

1. 목표와 작업 위치

위치	작업	결과
Ubuntu 빌드 호스트	ROS 레이어 추가, ARM64 크로스 빌드	ROS 2가 포함된 ext4 rootfs
Ubuntu 빌드 호스트	파일 검사, FWDN 패키징	`yu_ros2_d3g.fwdn.zip`
보드와 USB로 연결한 PC	FWDN 로더 전송, eMMC 기록	새 이미지가 기록된 보드
D3 보드	부팅, ROS 환경 설정, 노드 실행	talker/listener 동작 확인

`run-build.sh` 등은 **이번 수업에서 제공하는 보조 스크립트**다. Telechips SDK나 Yocto의 기본 명령이 아니다. 기존 `build/d3-g-topst-main` 대신 `build/d3-g-topst-ros2`에 새 이미지를 만든다. 기존 다운로드와 sstate 캐시는 공유하므로 삭제하지 않는다.

2. 이전 수업 환경 확인

다음 명령은 모두 **Ubuntu 빌드 호스트**에서 실행한다.

```
cd ~/d3-g-yocto
ls poky/oe-init-build-env
ls build/d3-g-topst-main/conf/{local.conf,bblayers.conf}
test -x ~/yocto-tools/tar-1.34/bin/tar && echo 'course tar ready'
df -h .
free -h
```

- SDK 위치는 `~/d3-g-yocto`로 가정한다. 현재 스크립트는 이 위치를 사용한다.
- 이전 수업의 TOPST 소스 리비전과 개인 포크 리비전은 `forks.json`에 기록돼 있다.
- 호스트는 Git, Python 3.10 이상, 기존 Yocto 빌드 의존성이 필요하다.
- `tar-1.34` 경로는 Ubuntu의 tar/pseudo 호환성 문제를 피하기 위한 수업 환경 설정이다. 없으면 먼저 호스트를 준비한다. 시스템 tar를 덮어쓰지 않는다.
- 다른 사람이 같은 SDK에서 빌드 중이면 설정이나 소스 리비전을 변경하지 않는다.

3. 학생용 저장소 다운로드

GitHub 로그인이 필요 없는 공개 저장소: https://github.com/yu-asw/automotive-sensor/tree/main/labs/ros2-yocto

```
cd ~
git clone --depth 1 --filter=blob:none --sparse \
  https://github.com/yu-asw/automotive-sensor.git automotive-sensor-labs
cd automotive-sensor-labs
git sparse-checkout set labs/ros2-yocto
git rev-parse HEAD
```

출력한 커밋을 실습 기록에 남긴다. 같은 이름의 디렉터리가 이미 있으면 새로 클론하지 말고 기존 파일과 변경 내용을 확인한다. 브라우저의 **Code → Download ZIP**으로 받아 압축을 풀어도 된다.

4. 스크립트와 수업용 레이어 설치

```
# 다운로드한 저장소의 루트에서
python3 labs/ros2-yocto/install-tools.py --check
python3 labs/ros2-yocto/install-tools.py
```

첫 명령은 설치할 파일과 전제조건만 검사한다. 두 번째 명령은 스크립트를 SDK 루트에, `meta-yu-ros2`를 `layers/` 아래에 배치한다. 기존 파일의 내용이 다르면 덮어쓰지 않고 중단한다. 이미 설치한 도구를 업데이트하는 경우에는 변경 사항을 검토하고 백업한 다음 교체한다.

5. meta-ros와 개인 포크 준비

이 단계는 **아직 ROS 빌드 설정을 만들지 않은 호스트에서 한 번** 실행한다.

```
cd ~/d3-g-yocto
mkdir -p layers
git clone --branch kirkstone https://github.com/ros/meta-ros.git layers/meta-ros
git -C layers/meta-ros checkout --detach \
  446dd9e3f26fe8a6594671b5b865872b4199f16c
python3 apply-forks.py
python3 prepare-build.py
```

이미 `layers/meta-ros`가 있으면 HEAD와 로컬 수정을 확인한다. 로컬 수정을 강제로 지우지 않는다. `apply-forks.py`는 고정된 Poky/meta-openembedded 개인 포크 커밋을 선택하고 repo local manifest를 만든다. 기존 기본 이미지·설정은 보존하지만 **소스 체크아웃은 개인 포크로 변경된다.** `prepare-build.py`는 기존 conf를 복사해 별도의 ROS 빌드 설정을 만들며 기존 ROS conf는 덮어쓰지 않는다.

이미 수업 준비를 마친 빌드 VM이라면

3-5단계를 다시 실행할 필요 없이 다음을 확인하고 7단계로 간다.

```
cd ~/d3-g-yocto
ls run-build.sh build-job.sh
cat build/d3-g-topst-ros2/conf/auto.conf
git -C layers/meta-ros rev-parse HEAD
```

이 조합은 기존 BSP의 Kirkstone을 유지한다. meta-ros Kirkstone은 지원 종료 상태이며, 최신 지원 조합으로 소개하지 않는다.

6. 8GB 호스트의 병렬 빌드 설정

이번 재빌드에서는 `build/d3-g-topst-ros2/conf/local.conf`의 **마지막**에 다음을 적용한다.

```
BB_NUMBER_THREADS = "1"
PARALLEL_MAKE = "-j 2"
PARALLEL_MAKE:pn-opencv = "-j 1"
PARALLEL_MAKE:pn-rust-llvm-native = "-j 1"
```

`local.conf`는 `auto.conf`보다 나중에 읽힌다. 기존 BSP의 `local.conf`에 강제 지정된 값이 있으면 `auto.conf`에서만 제한해도 실제 병렬도가 줄지 않는다. 새 준비 스크립트는 복사한 `local.conf` 끝에 제한을 추가한다. 빌드가 실행 중이지 않을 때 `./run-build.sh -e opencv`에서 최종 `BB_NUMBER_THREADS`, `PARALLEL_MAKE` 값을 확인한다.

지난 빌드는 OpenCV `cv2.cpp` 컴파일 중 `cc1plus`가 종료됐다. 커널 로그의 `Out of memory: Killed process ... (cc1plus)`로 메모리 부족을 확인했다. 동시에 무거운 레시피가 실행되는 것을 줄이고 OpenCV 자체도 한 프로세스로 컴파일한다. 이 변경을 위해 `tmp`나 다운로드 캐시를 삭제하지 않는다.

7. 이미지 빌드

```
cd ~/d3-g-yocto
./run-build.sh -n yu-ros2-image
./run-build.sh yu-ros2-image
```

첫 명령은 dry run이다. 실제 컴파일·부팅·카메라 동작을 검증하지 않는다. 두 번째 명령은 실제 빌드다. 실패한 뒤 원인을 수정했다면 동일 명령으로 이어서 진행한다.

SSH가 끊겨도 계속 빌드하려면 다음 방법을 사용한다. **위 빌드와 동시에 실행하지 않는다.**

```
cd ~/d3-g-yocto
nohup ./build-job.sh "$HOME/yocto-logs/ros2-lab-01" \
  > "$HOME/ros2-lab-01-launch.log" 2>&1 < /dev/null &
tail -f "$HOME/yocto-logs/ros2-lab-01/build.log"
```

`tail`은 Ctrl+C로 종료해도 빌드는 계속된다. 로그 디렉터리는 실행마다 새 이름을 사용한다. 빌드가 끝나면 확인한다.

```
cat ~/yocto-logs/ros2-lab-01/exit-code.txt
tail -20 ~/yocto-logs/ros2-lab-01/build.log
```

`exit-code.txt`의 `0`과 최종 오류 요약을 함께 본다. 파일이 없으면 아직 완료되지 않았을 수 있다.

8. 산출물 검증

```
cd ~/d3-g-yocto
DEPLOY="$PWD/build/d3-g-topst-ros2/tmp/deploy/images/d3-g-topst-main"
ls -lh "$DEPLOY"/yu-ros2-image*.manifest "$DEPLOY"/yu-ros2-image*.ext4
```

타임스탬프 파일과 심볼릭 링크가 함께 있을 수 있다. 같은 빌드의 manifest와 ext4를 선택한다. 이번 빌드에서 확인한 심볼릭 링크 이름은 다음과 같다. 다른 환경에서는 실제 생성 이름과 대조한다.

```
python3 verify-image.py \
  "$DEPLOY/yu-ros2-image-humble-d3-g-topst-main.manifest" \
  "$DEPLOY/yu-ros2-image-humble-d3-g-topst-main.ext4"
```

검사 항목은 ROS CLI·demo·카메라 패키지, `/opt/ros/humble`의 환경 스크립트와 실행 파일, 그리고 talker의 AArch64 ELF 형식이다. 이 단계는 보드 부팅 검사와 다르다.

9. 새 이미지의 FWDN 패키지 생성

원본 `stitch-fai-d3.sh`는 기존 빌드 경로를 참조한다. 새 ROS 이미지를 패키징할 때는 수업용 래퍼를 사용한다.

```
cd ~/d3-g-yocto
python3 package-fwdn.py "$DEPLOY/yu-ros2-image-humble-d3-g-topst-main.ext4" "$HOME/d3g-ros2-package-
01"
```

출력 디렉터리는 새 경로여야 한다. 결과:

```
~/d3g-ros2-package-01/
  inputs.json              새 rootfs와 부트 파일의 경로·SHA-256
  packaging.log            TOPST 패키징 로그
  yu_ros2_d3g.fwdn.zip      다운로드 PC로 가져갈 패키지
```

래퍼는 새 빌드의 입력 파일을 별도 작업 디렉터리에서 조합하고 ZIP CRC와 필수 파일을 확인한다. **보드에는 아무것도 쓰지 않는다.** 기존 `output_d3g.fwdn.zip`을 덮어쓰지 않는다.

```
cd ~/d3g-ros2-package-01
sha256sum yu_ros2_d3g.fwdn.zip > yu_ros2_d3g.fwdn.zip.sha256
```

10. 다운로드 PC로 복사

이제 명령을 실행하는 위치를 **보드와 USB로 연결할 PC**로 바꾼다.

```
# PC 터미널. 사용자와 주소는 실제 빌드 호스트 정보로 바꾼다.
scp <사용자>@<빌드호스트>:d3g-ros2-package-01/yu_ros2_d3g.fwdn.zip .
scp <사용자>@<빌드호스트>:d3g-ros2-package-01/yu_ros2_d3g.fwdn.zip.sha256 .
```

Linux에서는 `sha256sum -c yu_ros2_d3g.fwdn.zip.sha256`, macOS에서는 `shasum -a 256 -c yu_ros2_d3g.fwdn.zip.sha256`로 전송 후 일치 여부를 확인한다. Windows에서는 `Get-FileHash .\yu_ros2_d3g.fwdn.zip -Algorithm SHA256` 결과를 비교한다. 새 폴더에 압축을 풀고 `fwdn.sh` 또는 `fwdn.bat`이 **`yu_ros2_d3g.fai`**를 참조하는지 확인한다.

11. 보드를 FWDN 모드로 연결 — 장비가 있을 때

현재 보드가 없으므로 이 단계부터는 내일 실제 장비에서 진행한다.

1. D3-G 전원을 켠다.
2. Boot Mode 스위치를 누른 채 RESET을 한 번 누른다.
3. Boot Mode 스위치에서 손을 뗀다.
4. PC USB를 보드의 **FWDN 전용 USB Type-C 포트**에 연결한다.
5. 장치 인식을 확인한다. Linux의 Telechips USB vendor ID는 `140e`다.

시리얼 콘솔은 별도 UART 연결이며 115200 bps, 8N1, 흐름 제어 없음으로 준비한다. 일반 USB 허브 포트, 디버그 UART, FWDN 포트를 혼동하지 않는다.

12. 로더 전송, 장치 정보 확인, eMMC 기록

이 단계의 low-format과 쓰기는 보드의 기존 eMMC 내용을 지운다. 이전 수업 복구 이미지와 필요한 데이터를 먼저 보관한다. 처음에는 로더 전송으로 장치 정보를 확인한다. 로더는 RAM에서 동작하며 이 명령만으로 eMMC를 포맷하지 않는다.

```
# Linux: 새 ZIP을 푼 디렉터리에서
chmod +x fwdn
./fwdn --fwdn boot-firmware/fwdn.json
```

USB 권한 설정이 없으면 해당 FWDN 명령을 `sudo`로 실행한다. `Device info`, eMMC 용량과 DRAM 정보가 나온 뒤, 같은 세션에서 이어서 실행한다.

```
./fwdn --storage emmc --low-format
./fwdn -w boot-firmware/boot.single.json
./fwdn -w yu_ros2_d3g.fai --storage emmc --area user
```

Windows PowerShell은 같은 명령에서 `./fwdn` 대신 `.\fwdn.exe`를 사용한다. 전체 네 단계를 한 번에 실행하는 패키지 스크립트는 Linux `./fwdn.sh`, Windows `.\fwdn.bat`이다. 전체 스크립트 실행 전에는 보드를 다시 FWDN 모드로 준비한다.

macOS의 패키지 내 `fwdn`은 Linux 실행 파일이므로 그대로 실행할 수 없다. 교수자가 별도로 준비한 ARM64 macOS 포팅 도구를 사용하거나 Windows/Linux PC를 사용한다. macOS 포팅 도구는 실제 보드 검증이 아직 필요하며, 학생 공개 저장소에 포함하지 않는다.

성공 판정은 마지막 `Complete FWDN`과 명령 종료 상태다. 진행 중 전원·USB를 분리하지 않는다. 장치가 안 보이면 부트 모드·케이블·USB 권한을 확인한다. 노드가 없는 상태에서 포맷 명령을 반복하지 않는다.

13. 새 이미지 부팅과 ROS 2 확인

FWDN 완료 후 USB-C를 먼저 분리하고 전원을 끈다. Boot Mode를 누르지 않고 다시 전원을 넣는다. 시리얼에서 부팅 로그를 확인한 다음 **D3 보드 터미널**에서 실행한다.

```
cat /etc/os-release
uname -m
ip -br addr
. /opt/ros/humble/local_setup.sh
export ROS_DOMAIN_ID=10
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export ROS_LOCALHOST_ONLY=1
ros2 run demo_nodes_cpp talker
```

두 번째 보드 터미널에서 같은 환경을 설정하고 실행한다.

```
ros2 run demo_nodes_cpp listener
```

반복 수신을 확인한 뒤 `ros2 node list`, `ros2 topic list -t`, `ros2 topic info /chatter --verbose`를 확인한다. 보드-PC 통신 실습에서는 양쪽의 domain ID를 맞추고 `ROS_LOCALHOST_ONLY=0`으로 설정한다.

14. 제출할 검증 기록

- 학생 자료 커밋, 소스 manifest, 빌드 종료 코드와 마지막 로그
- 새 ext4와 FWDN ZIP의 SHA-256
- FWDN 장치 정보와 `Complete FWDN` 로그
- 부팅 후 OS·CPU·IP 정보와 talker/listener 송수신 결과

빌드 성공, 패키지 완성, eMMC 기록, 부팅, ROS 통신은 각각 따로 기록한다. USB 카메라 검증은 이후 V4L2 직접 캡처와 `/camera/image_raw` 확인으로 이어간다.

15. 2026-10-28 리허설 결과

검사	결과
build-07	8,354 tasks 성공, 종료 코드 0; 캐시 재사용을 포함한 이번 실행 26분 2초
rootfs	약 1.1GB; ROS CLI, demo, 카메라 패키지·실행 파일과 talker AArch64 형식 확인
FWDN ZIP	약 263MB; 파티션 크기 검사, ZIP CRC·필수 파일 검사, PC 전송 후 SHA-256 일치
QEMU 사용자 모드	talker 발행 로그 확인; 기본 및 UDP 전용 설정 모두 25초 내 listener 수신 미확인 — 통신 검증은 통과하지 못함
실물 보드·카메라	미연결; eMMC 기록·부팅·DDS 통신·카메라 캡처는 미검증

QEMU 결과만으로 원인을 보드 이미지 문제나 에뮬레이션 문제로 단정하지 않는다. 실제 보드에서 talker/listener를 먼저 검증한다. 빌드 경고는 기존 nng bbappend 대상 누락 1건과 camera-calibration-parsers / camera-info-manager의 BSD 라이선스 수집 경고 2건이다.

이번 rootfs SHA-256: `448a417693dd9aeb814bffcbb330a29e22653ad88af944932e33902b0a3cfc69`

이번 FWDN ZIP SHA-256: `9f62fdb9cc236f49f2f36e26b28f760d834a9232bbf4d6ffaa63f7fe930943dd`

이 해시는 이번 산출물의 식별값이며 학생이 재빌드한 이미지의 해시가 같아야 한다는 뜻은 아니다.

16. USB 카메라 실습 — 실제 장비에서 진행

준비

보드의 talker/listener 수신을 먼저 확인한다. PC에는 ROS 2 Humble과 `rqt_image_view`가 준비되어 있어야 한다. PC의 ROS 환경은 `/opt/ros/humble/setup.bash`, 이 Yocto 보드의 환경은 `/opt/ros/humble/local_setup.sh`로 설정한다. GUI 뷰어는 PC에서 실행한다.

PC와 보드의 각 터미널에서 팀별로 배정받은 동일한 `ROS_DOMAIN_ID`와 `RMW_IMPLEMENTATION=rmw_fastrtps_cpp`, `ROS_LOCALHOST_ONLY=0`을 설정한다. 예제 domain ID는 10이다. VPN이나 ping 성공만으로 DDS 검색·데이터 통신이 보장되지는 않는다.

A. 장치와 지원 포맷 확인 — D3 보드

```
lsusb
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --all
v4l2-ctl -d /dev/video0 --list-formats-ext
```

`/dev/video0`은 예시다. 실제 캡처 노드를 찾아 바꾼다. 카메라 하나가 여러 노드를 만들 수 있다. 이번 `v4l2_camera 0.6.2`는 YUYV, UYVY, GREY 입력을 지원한다. MJPEG만 지원하는 모드를 이 예제에 그대로 적용하지 않는다. 아래 명령은 **YUYV 640×480을 지원한다고 확인한 경우**에만 사용한다.

B. ROS 없이 캡처 — D3 보드

```
v4l2-ctl -d /dev/video0 \
  --set-fmt-video=width=640,height=480,pixelformat=YUYV \
  --stream-mmap --stream-count=30 \
  --stream-to=/tmp/camera-test.yuyv
```

종료 상태와 캡처 로그를 기록한다. 파일은 raw 프레임이며 MP4가 아니다. 파일 크기만으로 영상이 정상이라고 판정하지 않는다. 캡처 도구가 종료된 뒤 ROS 노드를 시작한다.

C. ROS 카메라 노드 — D3 보드

```
. /opt/ros/humble/local_setup.sh
export ROS_DOMAIN_ID=10
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export ROS_LOCALHOST_ONLY=0
ros2 run v4l2_camera v4l2_camera_node --ros-args \
  -r __ns:=/camera \
  -p video_device:=/dev/video0 \
  -p pixel_format:=YUYV \
  -p 'image_size:=[640,480]'
```

시작 로그에서 협상된 포맷과 오류를 확인한다. 두 번째 보드 터미널에서도 같은 ROS 환경을 설정하고 실행한다.

```
ros2 node list
ros2 topic list -t
ros2 topic info /camera/image_raw --verbose
ros2 topic hz /camera/image_raw
```

`topic hz`는 해당 구독자가 수신한 빈도다. 카메라 설정 FPS, 발행 빈도, 네트워크 수신 빈도, 뷰어 표시 FPS를 구분한다.

D. PC에서 보기

```
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=10
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export ROS_LOCALHOST_ONLY=0
ros2 run rqt_image_view rqt_image_view
```

`/camera/image_raw`를 선택한다. 영상이 없으면 USB 캡처 → 노드 로그 → 토픽 검색 → QoS 호환성 → 네트워크 순서로 확인한다. ROS 없이 캡처가 실패하면 domain ID 변경으로 해결되지 않는다.

E. 제출 기록

항목	학생 기록
팀 / 날짜 / 학생 자료 커밋	
이미지 파일명·SHA-256 / 보드 OS	
FWDN 결과 / 부팅 결과	
talker/listener 수신 결과	
카메라 모델 / 실제 video 노드	
FOURCC / 해상도 / 지원 frame interval	
ROS domain / 이미지 토픽 / publisher QoS	
관측 수신 빈도 / 측정 위치·시간	
PC 뷰어 화면 / 재실행 명령	
한 가지 설정 변경과 결과 / 원상 복구 결과	

실패하거나 장비가 없으면 상태와 로그를 그대로 기록한다. 관측하지 않은 FPS나 부팅 결과를 채워 넣지 않는다.

참고: 고정된 v4l2_camera README, ROS 2 Humble 문서.